

Autonomous Drone ROS 2 Setup Guide

Repository

`autonomous_drone_ros2`

Stack

ROS 2 Humble, PX4 SITL, Gazebo Harmonic, MAVROS, ArUco-based precision landing, FastAPI backend, React dashboard. The repository README describes a mission flow that goes from takeoff at A, to delivery points B/C/D, and then back home, with ArUco search, align, precision land, and payload drop at each point.

1. What this project contains

The repository is organized around these main parts:

- `src/autonomous_drone_ros2/`
- `drone_base/` — arm, disarm, takeoff, land
- `drone_camera/` — Gazebo camera bridge
- `drone_vision/` — ArUco ID=17 detection
- `drone_controller/` — waypoint navigation and ArUco landing
- `drone_mission/` — full mission orchestration
- `drone_navigation/` — Nav2 + SLAM placeholder
- `drone_interfaces/` — shared constants and topic names
- `drone_msgs/` — custom messages planned for later
- `drone_bringup/` — launch files
- `simulation/gazebo/`
- `worlds/` — `aruco_landing.sdf`
- `models/aruco_17/` — landing marker model and texture
- `scripts/`
- `generate_aruco.py`
- `setup_gazebo.sh`
- `build.sh`
- `docs/`
- architecture
- setup
- milestones
- API docs
- `docker/`
- deployment Dockerfile

The README also shows a planned workspace layout that includes `drone_dashboard` and `drone_backend` inside the ROS workspace.

2. System prerequisites

Install on Ubuntu 22.04 with ROS 2 Humble.

Required packages from the repository README:

```
sudo apt install ros-humble-mavros ros-humble-mavros-extras -y
sudo apt install ros-humble-cv-bridge python3-opencv -y
sudo apt install ros-humble-ros-gz-bridge -y
sudo apt install python3-colcon-common-extensions -y
```

Install GeographicLib datasets for MAVROS:

```
wget https://raw.githubusercontent.com/mavlink/mavros/master/mavros/scripts/
install_geographiclib_datasets.sh
sudo bash install_geographiclib_datasets.sh
```

The README explicitly states the project targets ROS 2 Humble and expects PX4-Autopilot to be cloned at `~/PX4-Autopilot`.

3. Clone the repository

```
mkdir -p ~/drone_ws/src
cd ~/drone_ws/src
git clone https://github.com/aadhi4200/autonomous_drone_ros2.git
```

Then build the workspace:

```
cd ~/drone_ws
source /opt/ros/humble/setup.bash
colcon build --symlink-install
source install/setup.bash
```

To auto-load ROS and the workspace in every terminal:

```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
echo "source ~/drone_ws/install/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

This matches the repository README setup flow.

4. Set up the Gazebo ArUco landing world

Generate the ArUco marker texture:

```
python3 ~/drone_ws/src/autonomous_drone_ros2/scripts/generate_aruco.py
```

Copy the world and marker model into PX4:

```
bash ~/drone_ws/src/autonomous_drone_ros2/scripts/setup_gazebo.sh
```

The README says the Gazebo world uses `aruco_landing.sdf` and the marker model is `aruco_17`.

5. PX4 + Gazebo SITL setup

The repository README originally showed a single `make px4_sitl gz_<model>` style workflow, but it also notes that on PX4 v1.18.0-alpha1 and later, that target no longer works and the project should instead be run as two separate processes. The README says to use `make px4_sitl_default`, then start Gazebo separately, then start PX4 separately with `-d`.

Build PX4 once

```
cd ~/PX4-Autopilot
make px4_sitl_default
```

Start Gazebo

```
export GZ_SIM_RESOURCE_PATH=~/PX4-gazebo-models/models:~/drone_ws/src/
autonomous_drone_ros2/simulation/gazebo/models
export GZ_SIM_SERVER_CONFIG_PATH=~/PX4-gazebo-models/server.config
gz sim -r ~/drone_ws/src/autonomous_drone_ros2/simulation/gazebo/worlds/
aruco_landing.sdf
```

Start PX4

```
cd ~/PX4-Autopilot/build/px4_sitl_default/rootfs
export PX4_SYS_AUTOSTART=4018
export PX4_GZ_WORLD=aruco_landing
~/PX4-Autopilot/build/px4_sitl_default/bin/px4 -d
```

The README specifically warns that `-d` matters because the PX4 shell can spin without it.

6. Start MAVROS

In a new terminal:

```
source /opt/ros/humble/setup.bash
ros2 launch mavros px4.launch fcu_url:="udp://:14540@127.0.0.1:14580"
```

The README says to wait until MAVROS reports the FCU connection as established.

7. Start the camera bridge

In another terminal:

```
source /opt/ros/humble/setup.bash
ros2 run ros_gz_bridge parameter_bridge
  '/world/default/model/x500_lidar_cam_down_0/link/camera_link/sensor/camera/
  image@sensor_msgs/msg/Image@gz.msgs.Image'
```

This bridges Gazebo camera data into ROS 2.

8. Start the drone ROS 2 nodes

```
source /opt/ros/humble/setup.bash
source ~/drone_ws/install/setup.bash
ros2 launch drone_bringup full_mission.launch.py
```

The repository README lists these nodes as the expected startup set:

- `camera_node`

- vision_node
- drone_base
- waypoint_nav
- aruco_landing
- mission_manager

The vision node is configured to detect ArUco ID=17.

9. Start the mission

```
source ~/drone_ws/install/setup.bash
ros2 topic pub /mission/command std_msgs/msg/String "data: 'START'" --once
```

Abort the mission with:

```
ros2 topic pub /mission/command std_msgs/msg/String "data: 'ABORT'" --once
```

The README says ABORT should stop navigation, abort landing, and trigger AUTO.LAND.

10. Monitor progress

Useful monitoring commands:

```
ros2 topic echo /mission/status
ros2 topic echo /drone_base/status
ros2 topic echo /waypoint_nav/status
ros2 topic echo /aruco_landing/status
ros2 topic echo /vision/landing_target
ros2 topic echo /mavros/local_position/odom
ros2 topic list
ros2 node list
```

These are listed in the repository README as the standard status checks.

11. Update waypoint coordinates

The README points to:

```
src/autonomous_drone_ros2/drone_controller/drone_controller/  
waypoint_navigator.py
```

Waypoints A/B/C/D are edited there, then the package is rebuilt:

```
cd ~/drone_ws  
colcon build --symlink-install --packages-select drone_controller
```

12. Verification checklist

Before considering the setup complete, confirm:

- PX4 SITL starts
- Gazebo opens with the `aruco_landing` world
- MAVROS connects to PX4
- Camera bridge creates the image topic
- All six ROS 2 nodes start cleanly
- Mission `START` is accepted
- Drone arms, takes off, flies to B
- Drone detects ArUco marker
- Drone lands precisely
- Drone completes C, D, and return-to-home

13. Notes from the repository

The repository README shows 28 commits and includes the files `README.md`, `CMakeLists.txt`, `LICENSE`, `CLAUDE.md`, `drone_ws1_setup.sh`, `launch_full_sim.sh`, `manual_6_terminals.sh`, plus zipped workspace, model, and world assets.

It also notes that `launch_full_sim.sh` and `full_stack.launch.py` already use the corrected two-process PX4/Gazebo startup.

14. Recommended next step

Run the repository's own launch scripts first if you want the fastest path to a working sim stack:

```
bash launch_full_sim.sh
```

or the manual terminal sequence above if you want to debug each subsystem separately.